

Algorithms & Data Structures (M)

3.1. ARRAY DATA STRUCTURE

Properties

Insertion

Deletion

Searching: linear search, binary search

Merging

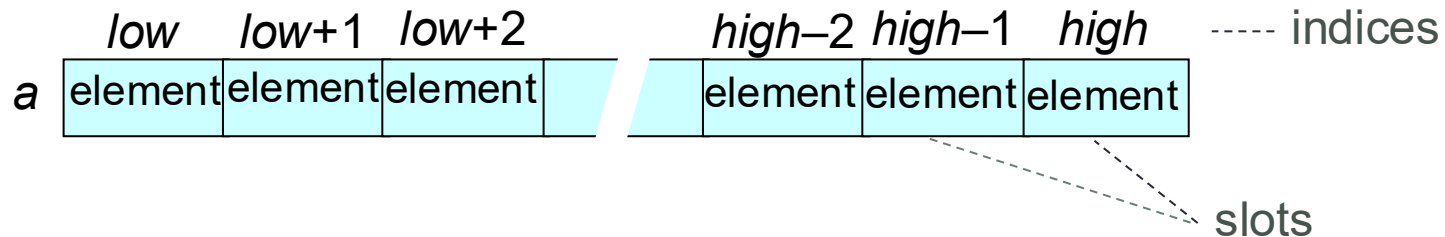
Sorting: selection sort, merge-sort, quick-sort

You've seen arrays in ProgIT, week 4 and lecture 2 week 9

Properties (1)

狭槽

- An **array** is a sequence of **slots**. Each slot contains an **element** (value or object).
- Each slot has a fixed **index**. The indices are consecutive integers.



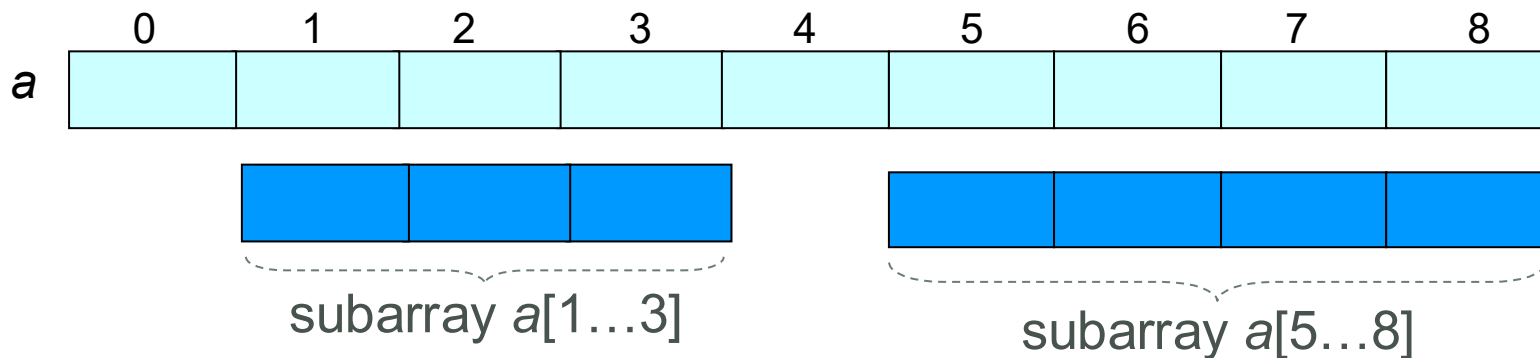
You have seen arrays in PrologIT

Properties (2)

- An array's **length** is the number of slots. The length is fixed when the array is constructed.
- Any array slot can be efficiently accessed (inspected or updated) using its index, in $O(1)$ time.

Properties: subarrays

- A **subarray** of an array a is a sequence of consecutive slots in a .
- Notation:
 - $a[l \dots r]$ is the subarray of a that comprises slots $a[l]$, $a[l+1]$, ..., $a[r]$.
 - Subarray notation may be used to express algorithms, but it is **not** directly supported by Java.



- Subarray $a[l \dots r]$ has length $r - l + 1$.

Properties: sorted arrays

- A (sub)array is **sorted** if its elements are in ascending order, i.e., each element is less than or equal to the element on its right.
- The meaning of “**less**” must be defined for each data type:
 - For **numbers**:
“x is less than y” means that x is numerically smaller than y, i.e., $x < y$.
 - For **strings**:
“x is less than y” conventionally means that x lexicographically precedes y. E.g.: “bat” is less than “bath”, which in turn is less than “bay”.

Aside: Java's Comparable interface (1)

- The following interface (from `java.util`) captures this notion:

```
public interface Comparable {  
    public int compareTo (Object that);  
    // Return a negative integer if this object is less than  
    // that, or zero if this object is equal to that, or a  
    // positive integer if this object is greater than that.  
}
```

Aside: Java's Comparable interface (2)

- If a class `Thing` implements `Comparable`, it must define the `compareTo` method in accordance with the above contract:

```
public class Thing implements Comparable {  
    ...  
    public int compareTo (Object that) {...}  
    // Return a negative integer if this thing is less than  
    // that, or zero if this thing is equal to that, or a  
    // positive integer if this thing is greater than that.  
}
```

- This captures the notion of “less” and “greater” for objects of class `Thing`.
- `String`, `Integer`, ... implement `Comparable`.

Aside: Java's Comparable<E> interface (2)

- As we saw in Proglt, there's also a **generic** version
- Much nicer because don't have to check "that" is a member of the right class
- In this course we will use generics a lot
- Having an array of objects is ok for now but
 - Harder
 - A bit old fashioned and clunky
 - Useful for heterogenous collections only
- Will do almost everything using generics (when appropriate) from now on.
 - You should revise the generics material from ProglT (or look at the pre-course revision slides on generics).

What if we want to sort with respect to different elements of a class at different times? Use Comparators, see example "SortExample" in arrays folder (code), see demo.

The Arrays class in Java

Java.util.Arrays Contains various (static) methods for manipulating arrays (e.g. cloning, copying, searching, sorting, for common types and generic type T ...) See <http://docs.oracle.com/javase/8/docs/api/> for details. E.g. int versions:

(Static) Method	Ret type	Description
<code>binarySearch(int[] a, int key)</code>	int	Searches specified array of ints for specified value using binary search algorithm
<code>copyOf(int[] original, int newLength)</code>	int[]	Copies specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
<code>sort(int[] a)</code>	void	Sorts the specified array into ascending numerical order.
<code>sort(T[] a, Comparator<T> c)</code>	void	Sorts the specified array with respect to the comparator c

Rest of this section we are going to implement (generic versions of) the insert, search and sort algorithms ourselves

Insertion (1)

- **Problem:** Given a (sub)array $a[\text{left} \dots \text{right}]$, insert an element $elem$ in the slot $a[\text{ins}]$. If necessary, shift elements to make way for it.

- **Array insertion algorithm:**

To insert $elem$ at index ins in $a[\text{left} \dots \text{right}]$
(where $\text{left} \leq ins \leq \text{right}$):

1. Copy $a[\text{ins} \dots \text{right}-1]$ into $a[\text{ins}+1 \dots \text{right}]$.
2. Copy $elem$ into $a[\text{ins}]$.
3. Terminate.

----- i.e., shift
elements one
slot to the right

Insertion (2)

- Analysis (counting copies):

Let $n = \text{right} - \text{left} + 1$ be the length of the array.

Step 1 performs between 0 and $n-1$ copies. On average it performs $(n-1)/2$ copies.

Step 2 performs 1 copy.

$$\begin{aligned}\text{Average no. of copies} &= (n - 1)/2 + 1 \\ &= n/2 + 1/2 \\ &\approx n/2\end{aligned}$$

Time complexity is $O(n)$.

1. Copy $a[\text{ins} \dots \text{right}-1]$ into $a[\text{ins}+1 \dots \text{right}]$.
2. Copy elem into $a[\text{ins}]$.
3. Terminate.

Deletion (1)

- Problem: Given a (sub)array $a[\text{left} \dots \text{right}]$, delete the element in the slot $a[\text{del}]$. If necessary, shift elements to fill the gap.
- **Array deletion algorithm:**
To delete the element at index del in $a[\text{left} \dots \text{right}]$ (where $\text{left} \leq \text{del} \leq \text{right}$):
 - 1. Copy $a[\text{del}+1 \dots \text{right}]$ into $a[\text{del} \dots \text{right}-1]$.
 - 2. Make $a[\text{right}]$ unoccupied.
 - 3. Terminate.

----- i.e., shift
elements one
slot to the left

Deletion (2)

- Analysis (counting copies):

Let $n = \text{right} - \text{left} + 1$ be the length of the array.

Step 1 performs between 0 and $n-1$ copies.

$$\begin{aligned}\text{Average no. of copies} &= (n - 1)/2 \\ &\approx n/2\end{aligned}$$

Time complexity is $O(n)$.

1. Copy $a[\text{del}+1 \dots \text{right}]$ into $a[\text{del} \dots \text{right}-1]$.
2. Make $a[\text{right}]$ unoccupied.
3. Terminate.

Searching

- **Problem:** Given a (sub)array $a[\textit{left} \dots \textit{right}]$, find which (if any) element equals a given target value.
- Choice of algorithms:
 - **linear search** (unsorted or sorted array)
 - **binary search** (sorted array).

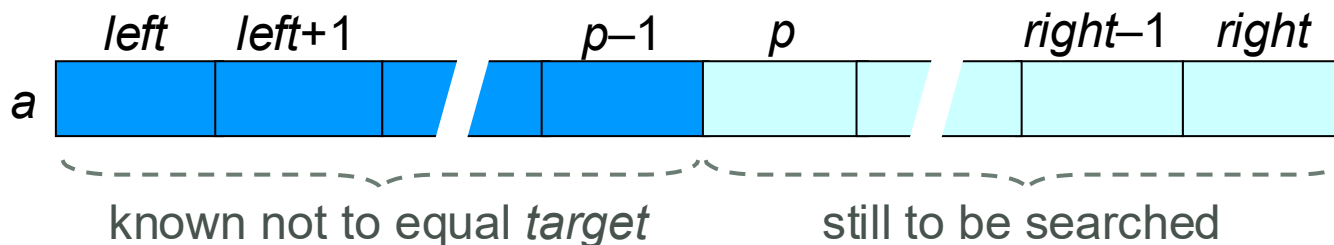
Linear search (1)

- **Linear search algorithm:**

To find which (if any) element of $a[\text{left} \dots \text{right}]$ equals target :

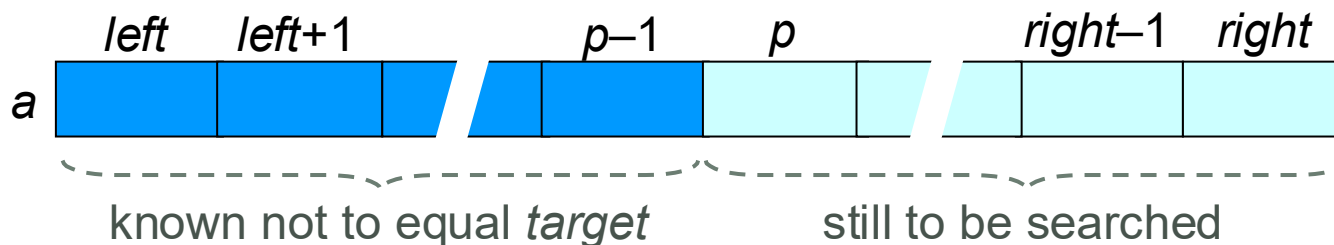
1. For $p = \text{left}, \dots, \text{right}$, repeat:
 - 1.1. If target equals $a[p]$, terminate yielding p .
2. Terminate yielding none .

- Loop invariant (state of play whenever enter loop):



Aside: invariants

- An **invariant** is an assertion that always holds at a particular step in an algorithm or program.
- In particular, a **loop invariant** is one that holds at every iteration of a loop.
- We'll use schematic diagrams to illustrate, e.g.:

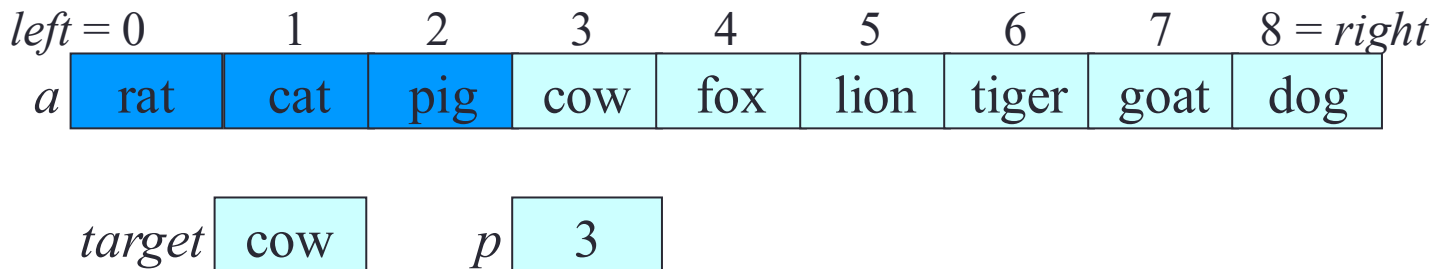


Linear search (2)

- Animation (successful search):

To find which (if any) element of $a[\text{left} \dots \text{right}]$ equals *target*:

1. For $p = \text{left}, \dots, \text{right}$, repeat:
 - 1.1. If *target* equals $a[p]$, terminate yielding p .
2. Terminate yielding *none*.



Linear search (3)

- Analysis (counting comparisons):

Let $n = \text{right} - \text{left} + 1$ be the length of the array.

- If the search is **unsuccessful**, step 1.1 is repeated n times.

No. of comparisons = n

- If the search is **successful**, step 1.1 is repeated between 1 and n times.

Average no. of comparisons = $(n + 1)/2$
 $\approx n/2$

- In either case, time complexity is $O(n)$.

1. For $p = \text{left}, \dots, \text{right}$, repeat:
 - 1.1. If *target equals* $a[p]$, terminate yielding p .
2. Terminate yielding *none*.

Linear search (4)

- Implementation in Java:

```
static <E> int linearSearch (E target,  
                             E[] a, int left, int right) {  
    // Find which (if any) element of a[left...right]  
    // equals target.  
    for (int p = left; p <= right; p++) {  
        if (target.equals(a[p]))  
            return p;  
    }  
    return -1;  
}
```

Binary search (1)

- Assume now that we are searching a **sorted** (sub)array.
- Consider searching a dictionary for a target word:
 - Bad idea: Look at page 1, then page 2, etc., until you find the page containing the target word.
This is linear search!
 - Better idea: Choose a page near the middle. If the target word happens to be on that middle page, you're finished. If the target word is less (greater) than the words on that middle page, focus on the pages before (after) that middle page, and repeat.
This is **binary search**.

Binary search (2)

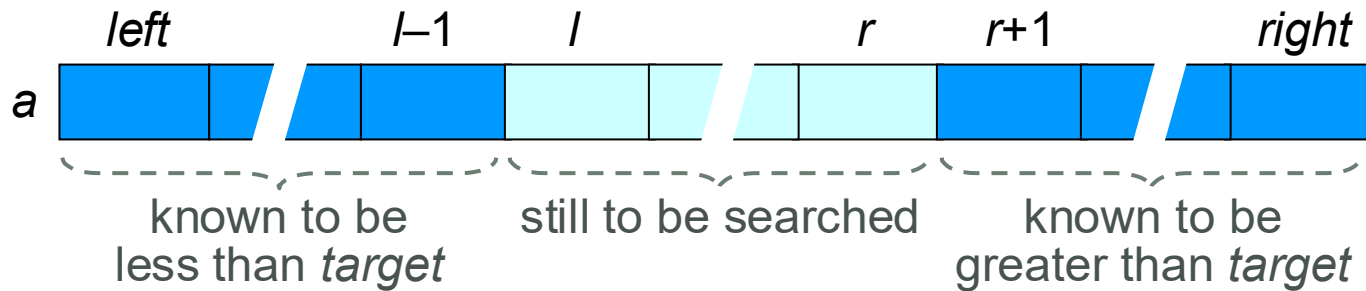
- **Binary search algorithm:**

To find which (if any) element of the sorted (sub)array $a[\textit{left} \dots \textit{right}]$ equals *target*:

1. Set $l = \textit{left}$, and set $r = \textit{right}$.
2. While $l \leq r$, repeat:
 - 2.1. Let m be an integer about midway between l and r .
 - 2.2. If *target* equals $a[m]$, terminate yielding m .
 - 2.3. Else, if *target* is less than $a[m]$, set $r = m - 1$.
 - 2.4. Else, if *target* is greater than $a[m]$, set $l = m + 1$.
3. Terminate yielding *none*.

Binary search (3)

- Loop invariant:

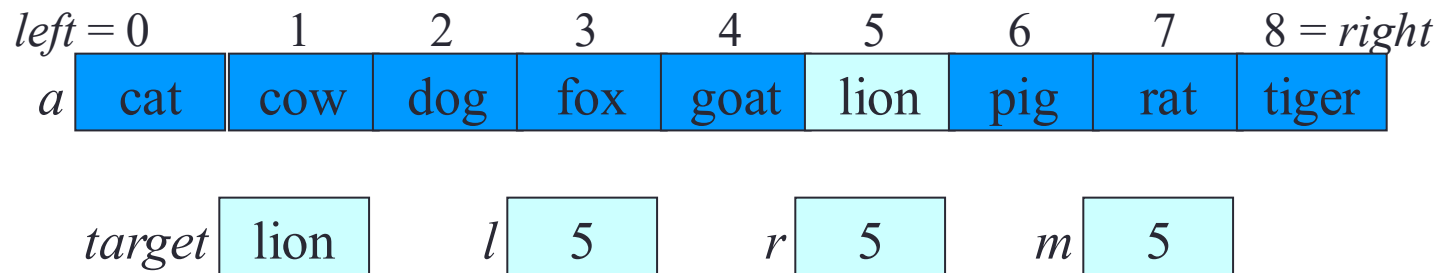


Binary search (4)

- Animation (successful search):

To find which (if any) element of the sorted (sub)array $a[\text{left} \dots \text{right}]$ equals *target*:

1. Set $l = \text{left}$, and set $r = \text{right}$.
2. While $l \leq r$, repeat:
 - 2.1. Let m be an integer about midway between l and r .
 - 2.2. **If *target* equals $a[m]$, terminate yielding m .**
 - 2.3. Else, if *target* is less than $a[m]$, set $r = m - 1$.
 - 2.4. Else, if *target* is greater than $a[m]$, set $l = m + 1$.
3. Terminate yielding *none*.



Binary search (5)

- Analysis (counting comparisons): n is length of the array. Assume steps 2.2–4 perform 1 comparison.
- If the search is **unsuccessful**, these steps are repeated as often as we must halve n to reach 0:

No. of comparisons $\approx \log_2 n + 1$

- If the search is **successful**, these steps are repeated *at most* that many times:

Max. no. of comparisons $\approx \log_2 n + 1$

- In either case, the time complexity is $O(\log n)$.

1. Set $l = \text{left}$, and set $r = \text{right}$.
2. While $l \leq r$, repeat:
 - 2.1. Let m be an integer about midway between l and r .
 - 2.2. **If $\text{target equals } a[m]$, terminate yielding m .**
 - 2.3. Else, if target is less than $a[m]$, set $r = m - 1$.
 - 2.4. Else, if target is greater than $a[m]$, set $l = m + 1$.
3. Terminate yielding *none*.

Binary search (6)

- Implementation in Java:

```

public static <E extends Comparable<E>> int
    binarySearch (E target,
        E[] a, int left, int right) {
    // Find which (if any) element of the sorted
    // (sub)array
    // a[left..right] equals target.
    int l = left, r = right;
    while (l <= r) {
        int m = (l + r) / 2; // roughly middle
        int comp = target.compareTo(a[m]);
        if (comp == 0)      return m;
        else if (comp < 0)  r = m - 1; // move right up
        else                l = m + 1; // move left down
    }
    return -1;
}

```

Comparison of searching algorithms

Algorithm	No. of comparisons	Time complexity
Linear search (unsorted array)	$\sim n/2$ (successful) n (unsuccessful)	$O(n)$
Linear search (sorted array)	$\sim n/2$	$O(n)$
Binary search (sorted array)	$\sim \log_2 n$	$O(\log n)$